

NLP PARA DETECCION DE PROMPT INJECTIONS

RESUMEN

- Propósito:

Diseñar un sistema de interceptación automatizada de Prompt Injections en interacciones con Modelos de Lenguaje de Gran Escala (LLMs), orientando el modelo hacia la máxima resiliencia operativa y protección de la capa de instrucciones del sistema.

- Metodología:

Se desarrolló un pipeline de NLP híbrido que integra vectorización TF-IDF de amplio espectro (unigramas y bigramas) con arquitectura de ingeniería de características estructurales. Se extrajeron métricas de densidad sintáctica (ratio de mayúsculas y frecuencia de caracteres especiales). Mediante una Regresión Logística con penalización balanceada para mitigar el sesgo de las clases de ataque minoritarias se creó un clasificador predictivo de prompts.

- Resultados principales:

El modelo alcanzó una precisión global del 75 % y un **Recall del 78 %** en la detección de inyecciones directas.

- Conclusiones:

La robustez del sistema reside en la combinación de semántica y sintaxis. Se demuestra que los ataques poseen patrones de ofuscación detectables.

INTRODUCCIÓN

La rápida integración de agentes de IA ha expuesto una vulnerabilidad sistémica: el *Prompt Injection*. Esto ocurre cuando entradas maliciosas manipulan el contexto del LLM para evadir restricciones de seguridad o exfiltrar datos. El estudio aborda el problema no como un fallo del modelo de lenguaje, sino como un desafío de clasificación de prompts. El mismo proporciona una metodología para segregar prompts benignos de aquellos con anomalías estructurales o intenciones adversarias claras. El objetivo es verificar la validez del proyecto, metodología y algoritmo empleado para detectar prompts maliciosos o dudosos.

METODOLOGÍA

El marco metodológico se diseñó para capturar tanto el significado como la forma de los ataques siguiendo un flujo de trabajo dividido en las siguientes etapas;

1. Ingesta y auditoría: Al dataset con 16012 registros, se le realizó una auditoría de integridad, ejecutando un mapeo de etiquetas basado en la severidad del riesgo (benigno, sospechoso e inyección). Se validó la consistencia de las clases
2. EDA: Se implementó una limpieza selectiva de texto que conserva símbolos estructurales o caracteres especiales (como :, [,], #) esenciales para identificar patrones de inyección pues éstos suelen delimitar bloques de comandos en ataques. Posteriormente, se aplicó una vectorización TF-IDF con Bigramas para capturar dependencias léxicas clave y secuencias de ataque ("ignore previous")

La vectorización TF-IDF penaliza términos comunes que carecen de valor discriminativo y resalta palabras clave de ataque que son raras en el corpus global pero frecuentes en documentos específicos de inyección. El uso de TF-IDF mejora la capacidad del modelo para identificar la "firma" del ataque frente al ruido del lenguaje natural.

3. Ingeniería de Características: Se crearon variables numéricas independientes para medir la "forma" del mensaje, extrayendo el ratio de mayúsculas (indicador de urgencia o "shouting") y la densidad de caracteres especiales (indicador de ofuscación o inyección de código/símbolos de escape).
4. Visualización y t-SNE 3D: Se empleó la técnica de reducción de dimensionalidad t-SNE 3D para proyectar los embeddings de los prompts. Esto permitió validar la separabilidad de las clases y detectar clústeres de anomalías
5. Modelado y entrenamiento: Se entrenó un modelo de Regresión Logística configurado `class_weight='balanced'`. Con ello se compensa la baja frecuencia estadística de los ataques en comparación con las interacciones legítimas, asegurando que el modelo no ignore las amenazas que pueden ser minoritarias.
6. Evaluación de resultados: Se realizó una auditoría enfocada en la matriz de confusión de seguridad, priorizando la reducción de falsos negativos (ataques no detectados) y analizando la curva de Precisión-Recall para determinar el umbral óptimo de activación del filtro.

RESULTADOS

El análisis de los hallazgos revela una capacidad de discriminación superior a los métodos de filtrado por palabras clave:

➔ RENDIMIENTO DEL MODELO:

El sistema no solo logró un **75 % de Accuracy**, sino que mantuvo un **F1-Score de 78 % para la clase suspicious**, demostrando que puede identificar intentos de sondeo que aún no son ataques frontales. No obstante, el parámetro clave en este proyecto es el **Recall, siendo el global del 78%**.

DISCUSIÓN

- Hallazgo estructural crítico:
Se descubrió que el `special_char_ratio` es el predictor más potente para ataques de "inyección de código" dentro del prompt, donde el atacante intenta inyectar sintaxis de Python o Markdown para romper el flujo lógico del LLM.
- Análisis de importancia:
Los coeficientes del modelo validaron que términos de "autoridad" como "system", "administrator" y "instruction" son señales de alerta máximas. Se detectó que los `prompt_injections` frecuentemente incluyen frases de tiempo verbal imperativo.
- Auditoría de Confianza:
Un análisis de falsos negativos mostró que el modelo tiene una confianza media-baja cuando se equivoca, lo que permite establecer un umbral de seguridad donde cualquier predicción por debajo del 85% de confianza se derive automáticamente a una supervisión humana o a un modelo de validación superior.
- Los resultados confirman que el *Prompt Injection* no es un problema puramente lingüístico, sino también de ingeniería de formato. La principal aportación para la toma de decisiones empresariales es la **estratificación del riesgo**: al clasificar las entradas como "sospechosas", las empresas pueden evitar el bloqueo total (que frustra al usuario) y optar por una monitorización activa. Este enfoque resuelve el dilema entre seguridad y usabilidad. Además, se observó que el modelo es resiliente ante ataques de "mayúsculas" (gritos), gracias a que el ratio de mayúsculas se normalizó como una característica independiente, evitando que el vectorizador de texto perdiera esta información. La limitación encontrada radica en ataques extremadamente cortos (menos de 10 palabras), donde la falta de contexto semántico dificulta la clasificación, sugiriendo que la longitud del mensaje es un factor condicionante de la seguridad.

CONCLUSIÓN

El estudio establece un estándar para la creación de capas de defensa perimetral en ecosistemas de IA generativa, como los agentes de IA o chatbots. La importancia de haber logrado un **Recall del 78%** en inyecciones subraya que el Machine Learning tradicional sigue siendo de gran valor en ciberseguridad debido a su rapidez y explicabilidad. Se recomienda la integración de este pipeline en arquitecturas que validen el prompt siempre antes de que alcance la API del LLM. Como línea de investigación futura, se propone el entrenamiento continuo, asegurando que el modelo evolucione a la par que las tácticas de los atacantes, manteniendo la integridad en los sistemas autónomos de IA.

Así mismo, se puede ampliar capas de seguridad multilingües, a detección en emoticonos y análisis de sentimientos con Machine Learning no supervisado.

REFERENCIAS

Caubet Fernández, J. (2026). *Ciberseguridad en IA Generativa y agentes autónomos, Nivel 1: Fundamentos y riesgos* [Curso en línea]. Barcelona Activa – Cibernàrium.

Caubet Fernández, J. (2026). *I4016_Aula Virtual_Ciberseguridad en la Industria 4.0* [Curso en línea]. Barcelona Activa – Cibernàrium.

Gasull Jolís, J. (2026). *Python: Análisis de textos, temas y sentimientos* [Curso en línea]. LinkedIn Learning.

Kowsari, K., et al. (2019). Text classification algorithms: A survey. Information. https://doi.org/10.3390/info10040150

Passionfruit Pulse. (2025). *AI prompt engineering for accurate citations: A complete guide*. <https://www.passionfruitpulse.com/ai-prompt-engineering-accurate-citations> ([passionfruitpulse.com](https://www.passionfruitpulse.com/ai-prompt-engineering-accurate-citations) in Bing)

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, É. (2011). *Scikit-learn: Machine learning in Python*. Journal of Machine Learning Research, 12, 2825–2830.

Salton, G., & Buckley, C. (1988). *Term-weighting approaches in automatic text retrieval*. Information Processing & Management, 24(5), 513–523. [https://doi.org/10.1016/0306-4573\(88\)90021-0](https://doi.org/10.1016/0306-4573(88)90021-0)

Van der Maaten, L., & Hinton, G. (2008). *Visualizing data using t-SNE*. Journal of Machine Learning Research, 9, 2579–2605.